

Festival

Nayra está en un festival jugando a un juego donde el gran premio es un viaje a Laguna Colorada. El juego consiste en usar fichas para comprar cupones. Comprar cupones puede causar que consiga más fichas. El objetivo es obtener el máximo número de cupones.

Ella comienza el juego con A fichas, que puede usar para comprar cupones. Hay N cupones, numerados del 0 al $N - 1$. Nayra tiene que pagar $P[i]$ ($0 \leq i < N$) fichas para poder comprar el cupón i (y debe tener al menos $P[i]$ fichas antes de la compra). Puede comprar cada cupón como máximo una vez.

Además, a cada cupón i ($0 \leq i < N$) tiene un **tipo**, denotado por $T[i]$, que es un número entero **entre 1 y 4, inclusive**. Después de que Nayra compre el cupón i , el número de fichas restante se multiplica por $T[i]$. Formalmente, si tiene X fichas en algún momento del juego, y compra el cupón i (cosa que requiere $X \geq P[i]$), entonces tendrá $(X - P[i]) \cdot T[i]$ fichas después de la compra.

Tu tarea es determinar qué cupones debe comprar Nayra y en qué orden, para maximizar el número total de **cupones** que tiene al final. Si hay más de una secuencia de compras que logre esto, puedes reportar cualquiera de ellas.

Detalles de Implementación

Debes implementar el siguiente procedimiento.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : el número inicial de fichas que tiene Nayra.
- P : un array de longitud N que especifica los precios de los cupones.
- T : un array de longitud N que especifica los tipos de cupones.
- Este procedimiento se llama exactamente una vez para cada caso de prueba.

El procedimiento debe devolver un array R , que especifica las compras de Nayra de la siguiente manera:

- La longitud de R debe ser el número máximo de cupones que Nayra puede comprar.

- Los elementos del array son los índices de los cupones que debe comprar, en orden cronológico. Es decir, compra primero el cupón $R[0]$, después el cupón $R[1]$, y así sucesivamente.
- No debe haber elementos repetidos en R .

Si no se puede comprar ningún cupón, R debe ser un array vacío.

Restricciones

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ por cada i tal que $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ por cada i tal que $0 \leq i < N$.

Subtareas

Subtarea	Puntuación	Restricciones Adicionales
1	5	$T[i] = 1$ por cada i tal que $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ por cada i tal que $0 \leq i < N$.
3	12	$T[i] \leq 2$ por cada i tal que $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra puede comprar todos los N cupones (en algún orden).
6	16	$(A - P[i]) \cdot T[i] < A$ por cada i tal que $0 \leq i < N$.
7	18	Sin restricciones adicionales.

Ejemplos

Ejemplo 1

Considera la siguiente llamada:

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra tiene inicialmente $A = 13$ fichas. Ella puede comprar 3 cupones siguiendo el orden mostrado a continuación:

Cupón comprado	Precio del cupón	Tipo de cupón	Número de fichas después de la compra
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

En este ejemplo, no es posible que Nayra compre más de 3 cupones, y la secuencia de compras descrita anteriormente es la única forma en que puede comprar 3 cupones. Por lo tanto, el procedimiento debe devolver $[2, 3, 0]$.

Ejemplo 2

Considera la siguiente llamada:

```
max_coupons(9, [6, 5], [2, 3])
```

En este ejemplo, Nayra puede comprar ambos cupones en cualquier orden. Por lo tanto, el procedimiento debe devolver $[0, 1]$ o $[1, 0]$.

Example 3

Consider the following call.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

En este ejemplo, Nayra tiene una ficha, que no es suficiente para comprar ningún cupón. Por lo tanto, el procedimiento debería devolver $[]$ (un array vacío).

Grader de Ejemplo

Formato de entrada:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato de salida:

```
S  
R[0]  R[1]  ...  R[S-1]
```

Aquí, S es la longitud del array R devuelto por `max_coupons` .